

Universidade Estadual de Campinas - UNICAMP
Instituto de Computação
Introdução ao Processamento de Imagem Digital (MC920 / MO443)

Trabalho 2

Aluno: Pedro Brasil Barroso - RA 260637

Professor: Dr. Hélio Pedrini

Campinas
1º Semestre de 2025

Sumário

1	Introdução	2
2	Estrutura do trabalho	2
2.1	Requisitos	2
2.2	Organização dos arquivos	2
2.3	Como executar os exercícios	3
3	Exercício 1 - Técnicas de Meio Tom	4
3.1	Introdução	4
3.2	Metodologia	5
3.2.1	Arquitetura de Implementação	5
3.2.2	Estratégia de varredura	5
3.2.3	Aplicação da Difusão de Erro	7
3.3	Comparação entre RGB e escala de cinza	8
3.4	Resultados	9
3.5	Limitações	13
3.6	Conclusão	13
4	Exercício 2 - Filtragem e compressão no Domínio da Frequência	14
4.1	Filtragem	14
4.1.1	Introdução	14
4.1.2	Metodologia	15
4.1.3	Resultados	15
4.1.4	Limitações	20
4.2	Compressão	21
4.2.1	Introdução	21
4.2.2	Metodologia	21
4.2.3	Resultados	21
4.2.4	Limitações	23
4.3	Conclusão	23
5	Referências	23

1 Introdução

O objetivo deste trabalho é aplicar e analisar a técnica de meios-tons e a filtragem e compressão no domínio da frequência, explorando suas possibilidades de implementação, suas limitações práticas e os fundamentos teóricos nas quais se baseiam.

2 Estrutura do trabalho

2.1 Requisitos

As versões de Python e das bibliotecas utilizadas no projeto são as que seguem:

- Python: 3.12.3
- NumPy: 2.2.5
- OpenCV: 4.11.0.86
- Matplotlib: 3.10.1

2.2 Organização dos arquivos

O arquivo .zip submetido apresenta a seguinte organização (dentro do diretório `t2/`):

- `images/`
Contém as imagens de entrada utilizadas nos exercícios.
- `results/`
Armazena as imagens processadas (resultados).
- `helper_functions.py`
Script com funções auxiliares, que permitem salvar, ler e obter o caminho de imagens.
- `error_dispersion_matrices.py`
Script com a definição das matrizes de dispersão de erro utilizadas no exercício 1, as quais podem ser carregadas por meio da função `get_matrices`.
- `exercise1.py`
Script para executar o exercício 1.
- `exercise2_filtering.py`
Script para executar a filtragem do exercício 2.
- `exercise2_compression.py`
Script para executar a compressão do exercício 2.
- `requirements.txt`
Lista de dependências do projeto (bibliotecas Python necessárias).

2.3 Como executar os exercícios

Os scripts `exercise1.py`, `exercise2_filtering.py` e `exercise2_compression.py` são responsáveis pela execução dos exercícios presentes nesse trabalho. Ele deve ser executado via terminal com os seguintes argumentos:

- Argumentos comuns a todos os exercícios:
 - `-i`: Nome da imagem (localizadas no diretório `images`) a ser processada. Caso a seja especificada, é utilizada uma imagem padrão.
 - `-s`: Flag. Salva as imagens resultantes no diretório `results`. As imagens são salvas de acordo com o seguinte padrão:
`<numero_exercicio>/ex<numero>_<titulo>[_<identificador>].png`
Exemplos: `'01/ex01_meios_tons_burkes.png'`, `'02/ex02_comprimido.png'`
 - `-d`: Flag. Exibe as imagens processadas. Para interromper o programa, basta pressionar “q”.
- `-m` (apenas Exercício 1): Define quais matrizes de difusão de erro serão utilizadas (`'floyd_steinberg'`, `'stevenson_arce'`, `'burkes'`, `'sierra'`, `'stucki'`, `'jarvis_judice_ninke'` ou `'custom'`). Se não for definido, todas são processadas.
- Exercício 2 - Filtragem:
 - `-t`: Tipos de filtros (`'ideal'`, `'butterworth'` ou `'gaussian'`). Se não for definido, todos são processados.
 - `-m`: Modos dos filtros (`'lowpass'`, `'highpass'`, `'bandpass'` ou `'bandstop'`). Se não for definido, todos são processados.
 - `-c`: Frequência de corte (se aplica a todos os filtros).
 - `-w`: Largura dos filtros passa-faixa e rejeita-faixa.
 - `-o`: Ordem do filtro de Butterworth.
- `-p` (apenas Exercício 2 - compressão): Define qual o percentil inferior de frequências que serão removidas.

Exemplos de uso:

```
python3 exercise1.py -i seagull.png -m burkes sierra -d -s
```

```
python3 exercise2_filtering.py -i seagull.png -t butterworth gaussian \  
-m lowpass bandpass -c 50 -w 25 -o 3 -d -s
```

```
python3 exercise2_compression.py -i seagull.png -p 99.9 -d -s
```

3 Exercício 1 - Técnicas de Meio Tom

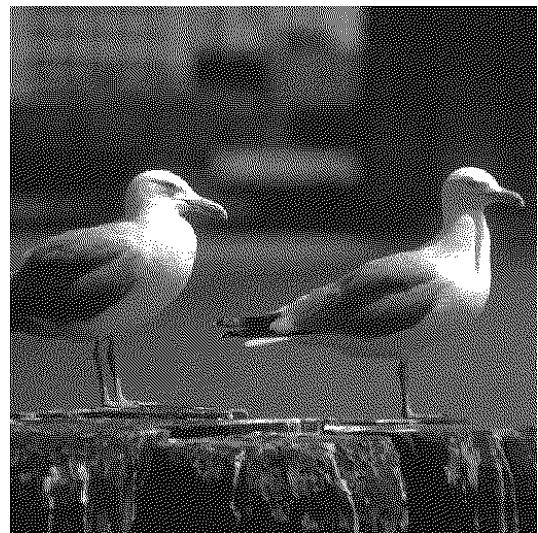
3.1 Introdução

A técnica de pontilhado com difusão de erros para confecção do efeito de meios-tons, conforme descrito por [1], consiste na binarização de imagens com propagação do erro de quantização para pixels próximos ainda não processados. A operação é realizada sequencialmente sobre cada pixel da imagem, utilizando uma matriz de pesos pré-definida para distribuir esse erro. A abordagem busca preservar a média local da intensidade original, o que resulta, visualmente, na ilusão de níveis intermediários de cinza, apesar das intensidades binárias (todos os pixels são pretos ou brancos). O mesmo pode ser realizado para imagens multibanda.

Um exemplo da aplicação desse método pode ser visto na Figura 1.



(a) Imagem original



(b) Imagem pontilhada

Figura 1 – Resultado da aplicação do método de Floyd-Steinberg

Considere uma imagem de intensidade $f(x, y)$ e uma matriz de difusão M com n elementos $(\Delta x_i, \Delta y_i, w_i)$ representando os deslocamentos - relativos à posição sendo processada (x, y) - e os respectivos pesos. O algoritmo de difusão de erro pode, então, ser resumido pela seguinte sequência de passos:

- O valor do pixel é binarizado: $g(x, y) = \text{limiar}(f(x, y))$
- O erro de quantização é calculado: $e(x, y) = f(x, y) - g(x, y)$
- O erro é distribuído: $f(x + \Delta x_i, y + \Delta y_i) += w_i \cdot e(x, y)$, para $i = 1, 2, \dots, n$

Vale citar que o somatório dos pesos $\sum_{i=1}^n w_i$, em todas as matrizes analisadas, é igual a 1, a fim de garantir a preservação da média de intensidade ao longo da imagem.

Além disso, na implementação que será analisada, a transformação de arredondamento $\text{limiar}(f(x, y))$ utilizada foi uma limiarização de intensidade, dada por:

$$g(x, y) = \begin{cases} 0, & \text{se } f(x, y) < 128 \\ 1, & \text{se } f(x, y) \geq 128 \end{cases}$$

É importante ressaltar que o cálculo do erro em uma determinada posição, durante a varredura, depende do valor (já modificado) dos pixels processados anteriormente; por conta disso, técnica não é paralelizável de forma completa. A não ser que se considere abordagens por blocos como em [2], o algoritmo permanece essencialmente sequencial.

Neste exercício, a técnica foi aplicada utilizando diferentes matrizes de difusão, a fim de analisar seus efeitos sobre o resultado final, bem como seus impactos na performance da implementação.

3.2 Metodologia

3.2.1 Arquitetura de Implementação

A definição da matriz de difusão de erro de Floyd-Steinberg é a que segue:

$$M^{FS} = \begin{bmatrix} & f(x, y) & 7/16 \\ 3/16 & 5/16 & 1/16 \end{bmatrix}$$

Note que duas das posições não precisam ser utilizadas ao difundir o erro $e(x, y) : M_{11}^{FS}$ e M_{12}^{FS} (posição vazia e a que contém $f(x, y)$, respectivamente), fato presente também - em mais posições - nas outras matrizes. Em termos de implementação, as matrizes analisadas foram definidas com o valor 0 nessas posições, de forma que a multiplicação $w \cdot e(x, y)$ resulta em zero e elas não são alteradas.

A escolha mencionada possibilitou a criação de uma função capaz de receber uma matriz de difusão de erro arbitrária, permitindo a flexibilidade no uso de diferentes matrizes. A seguir, apresentamos a implementação da matriz M^{FS} utilizada:

```
1 FLOYD_STEINBERG = np.array([
2     [0, 0, 7],
3     [3, 5, 1]
4 ], dtype=np.float16) / 16
```

Além disso, optou-se pelo tipo de dado *float16* - em detrimento de *float32* ou maior - para os cálculos durante a aplicação do método, uma vez que não há necessidade, neste processo, de grande precisão de ponto flutuante, e o valor máximo possível com essa representação (65,504) é mais do que suficiente para os cálculos necessários. Outras vantagens dessa escolha incluem economia de memória e aumento de performance.

3.2.2 Estratégia de varredura

A estratégia proposta por Floyd-Steinberg realiza uma varredura da esquerda para a direita, de cima para baixo. Essa abordagem faz com que o erro seja disperso sempre no mesmo sentido (para a direita e para baixo); uma alternativa, que visa contornar esse efeito,

é alternar a direção horizontal de varredura a cada linha (varredura em serpentina), mas a dispersão ainda é feita verticalmente apenas para baixo.

A diferença no resultado de cada uma dessas abordagens, embora visualmente parecida, pode ser visualizada na Figura 2. Note, na parte inferior do centro da esfera, que os pixels brancos seguem o formato dela na varredura em serpentina, mas formam uma distorção na varredura da esquerda para a direita. Contudo, a essa diferença é mais difícil de ser percebida visualmente para imagens usuais, como pode ser observado na Figura 3. Apesar disso, os resultados de ambas as Figuras apresentaram índice de Jaccard próximo de 0,65, i.e., apenas 65% de seus pixels são iguais.

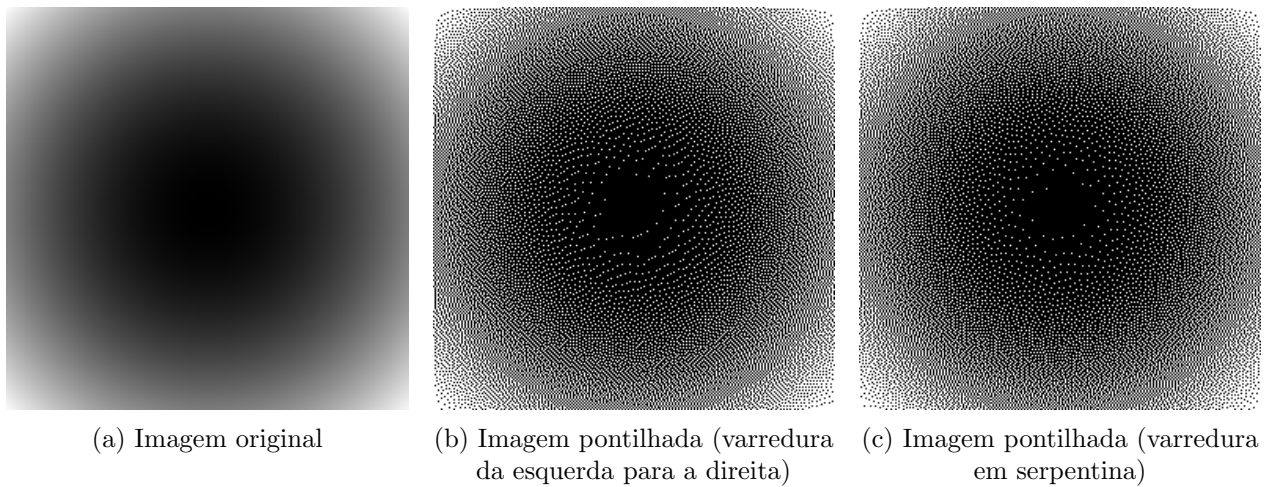


Figura 2 – Resultado da aplicação do método de Floyd-Steinberg com diferentes estratégias de varredura na imagem `esfera_256.png`

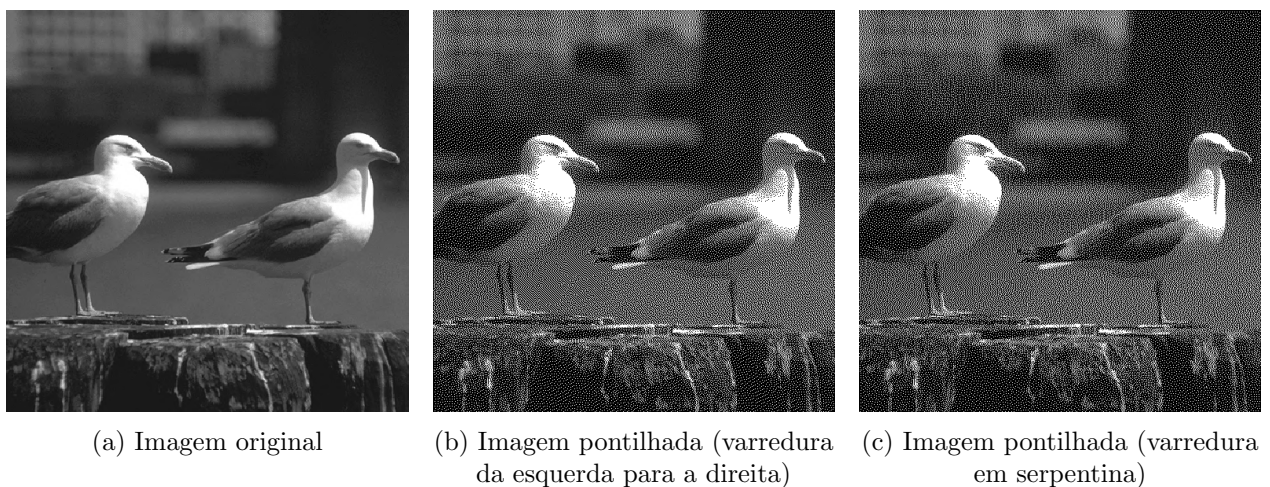


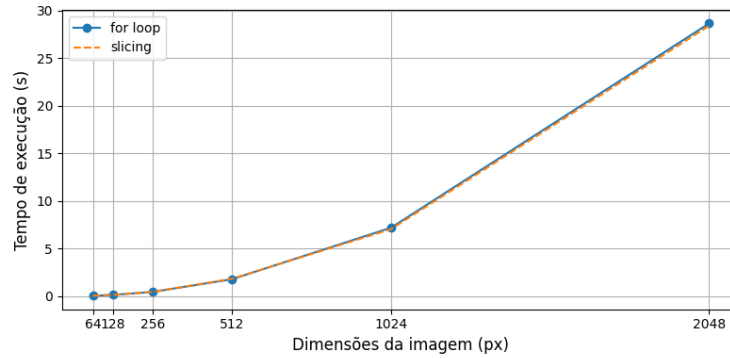
Figura 3 – Resultado da aplicação do método de Floyd-Steinberg com diferentes estratégias de varredura na imagem `seagull.png`

3.2.3 Aplicação da Difusão de Erro

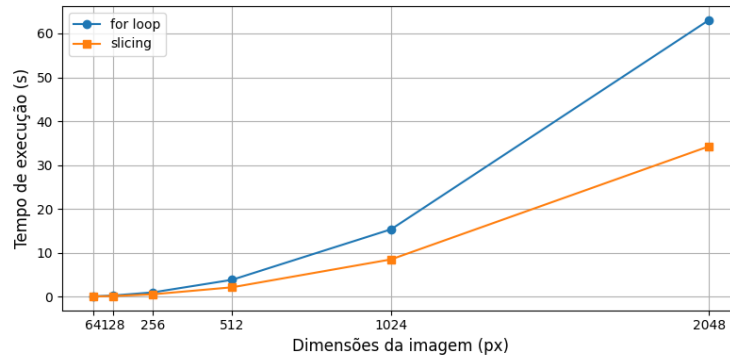
Inicialmente, a difusão de erro (de um pixel para seus vizinhos) foi implementada por meio de um *for loop* explícito, no qual o erro era distribuído de forma serial conforme os pesos definidos na matriz de dispersão. Esse algoritmo apresenta complexidade $O(H \cdot W \cdot h \cdot w)$, em que H e W representam as dimensões da imagem, e h e w , as dimensões da matriz.

Porém, uma vez que o erro foi calculado na posição (x, y) , ele pode ser distribuído de forma paralela à vizinhança. Para alcançar esse fim, é efetuado, na implementação, um *slicing* responsável por selecionar a região da imagem que está sendo modificada. A operação vetorial correspondente — a multiplicação do erro pela matriz de dispersão, seguida da soma com a subimagem — é automaticamente paralelizada pela biblioteca NumPy. Essa abordagem reduz a complexidade efetiva do algoritmo para $O(H \cdot W)$, assumindo um processador com capacidade ideal de paralelismo e paralelização completa pelo NumPy.

Na prática, conforme ilustrado na Figura 4, a implementação baseada em *slicing* é a mais eficiente para imagens RGB, apresentando *speedup* (aceleração) de aproximadamente duas vezes no processamento de imagens com dimensões grandes. Para imagens em escala de cinza, contudo, a diferença de desempenho entre as abordagens é desprezível.



(a) Escala de cinza (as curvas se sobrepõem)



(b) RGB

Figura 4 – Tempo de execução da aplicação da técnica de pontilhado por difusão de erro com as abordagens de *for loop* e *slicing*, utilizando a matriz de Floyd-Steinberg em imagens com diferentes dimensões.

3.3 Comparação entre RGB e escala de cinza

A técnica de pontilhado por difusão de erro pode ser estendida a imagens coloridas aplicando o algoritmo separadamente a cada canal (R, G, B). Em imagens em tons de cinza, o resultado final contém apenas dois valores de intensidade: preto e branco. No caso de imagens RGB, a combinação binária dos três canais leva a um total de $2^3 = 8$ cores distintas possíveis na imagem final.

Apesar de o número absoluto de cores ser maior em RGB, a redução relativa do espaço de cores é muito maior. A imagem original RGB possui até $255^3 = 16.581.375$ combinações possíveis de cor. Após o pontilhado, esse número cai para apenas 8:

$$\frac{2^3}{255^3} = 4,82 \cdot 10^{-7}$$

Para comparação, no caso de uma imagem em tons de cinza, a redução é de 255 para 2, o que equivale a:

$$\frac{2}{255} \approx 7,84 \cdot 10^{-3}$$

Apesar disso, o pontilhado de imagens coloridas ainda resulta em boa fidelidade visual em relação à imagem original.

Além disso, devido ao maior número de canais, imagens RGB demoram mais para ser processadas, como se nota na Figura 5, embora a otimização discutida na seção 3.2.3 diminua significativamente a diferença de tempo.

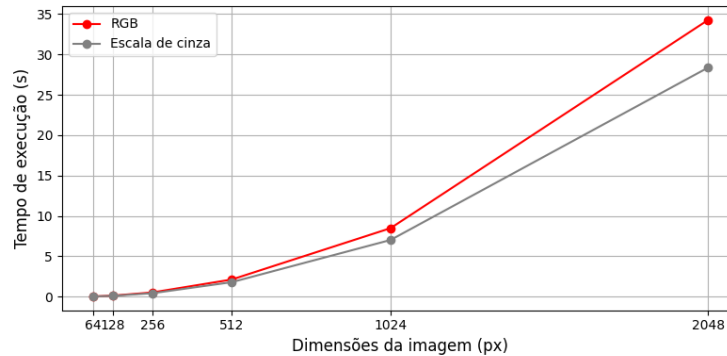


Figura 5 – Tempo de execução da aplicação da técnica de meios-tons por difusão de erro com a abordagem de *slicing*, utilizando a matriz de Floyd-Steinberg em imagens com diferentes dimensões.

Na Figura 6, é possível observar o efeito de pontilhado na mesma imagem em RGB e em escala de cinza. Devido à maior possibilidade de cores, o pontilhado da imagem RGB possui maior contraste e, portanto, mais detalhes de textura (esse fato é mais evidente nas folhas e no mar, localizados no *foreground* da imagem). Além disso, a correlação obtida para a imagem RGB foi de 0,51, enquanto, para a cinza, esse valor foi de 0,45; portanto, o pontilhado na imagem multibanda foi capaz de capturar de forma mais fidedigna as características da imagem original.

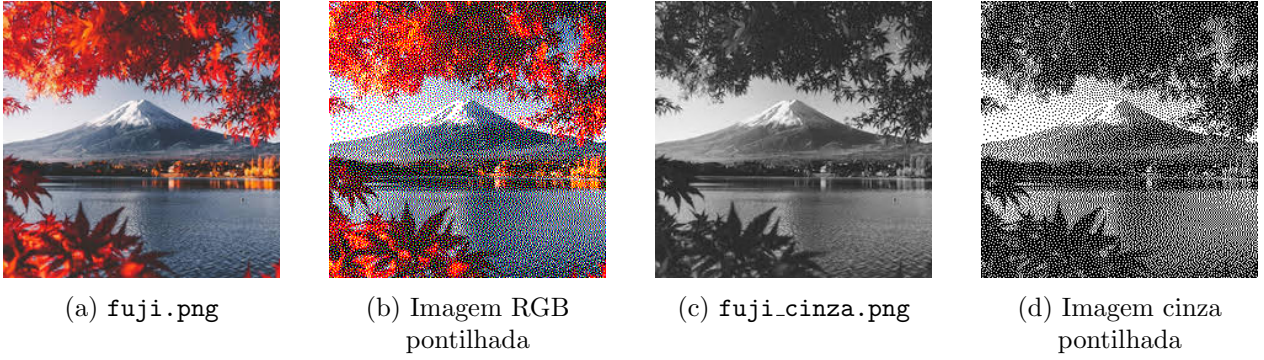


Figura 6 – Resultados da aplicação do método de Floyd-Steinberg sobre as imagens `fuji.png` (RGB) e `fuji_cinza.png` (escala de cinza)

3.4 Resultados

Nessa seção, a técnica de meios-tons foi aplicada com diferentes matrizes de dispersão de erro a três imagens: `fuji_cinza.png`, `degrade_radial.png` e `monalisa.png`, com tamanhos 225x225 (1 canal), 1024x1024 (1 canal) e 256x256 (3 canais), respectivamente.

Além das matrizes definidas no enunciado, também foi aplicada uma matriz como a de Floyd-Steinberg, porém com todos os pesos iguais a $\frac{4}{16}$, a fim de analisar a diferença no efeito visual. Nas Figuras, seus resultados têm a legenda “Customizado”.

$$\begin{array}{ll}
 \text{Floyd-Steinberg: } \begin{bmatrix} \frac{3}{16} & f(x,y) & \frac{7}{16} \\ & \frac{5}{16} & \frac{1}{16} \end{bmatrix} & \text{Customizado } \begin{bmatrix} & f(x,y) & \frac{4}{16} \\ \frac{4}{16} & \frac{4}{16} & \frac{4}{16} \end{bmatrix} \\
 \\
 \text{Burkes: } \begin{bmatrix} & f(x,y) & \frac{8}{32} & \frac{4}{32} \\ \frac{2}{32} & \frac{4}{32} & \frac{8}{32} & \frac{4}{32} \end{bmatrix} & \text{Sierra: } \begin{bmatrix} & f(x,y) & \frac{5}{32} & \frac{3}{32} \\ \frac{2}{32} & \frac{4}{32} & \frac{5}{32} & \frac{3}{32} \end{bmatrix} \\
 \\
 \text{Stucki: } \begin{bmatrix} & f(x,y) & \frac{8}{42} & \frac{4}{42} \\ \frac{2}{42} & \frac{4}{42} & \frac{8}{42} & \frac{4}{42} \\ \frac{1}{42} & \frac{2}{42} & \frac{4}{42} & \frac{1}{42} \end{bmatrix} & \text{Jarvis, Judice e Ninke: } \begin{bmatrix} & f(x,y) & \frac{7}{48} & \frac{5}{48} \\ \frac{3}{48} & \frac{5}{48} & \frac{7}{48} & \frac{5}{48} \\ \frac{1}{48} & \frac{3}{48} & \frac{5}{48} & \frac{1}{48} \end{bmatrix} \\
 \\
 \text{Stevenson e Arce: } \begin{bmatrix} & f(x,y) & \frac{32}{200} \\ \frac{12}{200} & \frac{26}{200} & \frac{30}{200} & \frac{16}{200} \\ & \frac{12}{200} & \frac{26}{200} & \frac{12}{200} \\ \frac{5}{200} & \frac{12}{200} & \frac{12}{200} & \frac{5}{200} \end{bmatrix}
 \end{array}$$

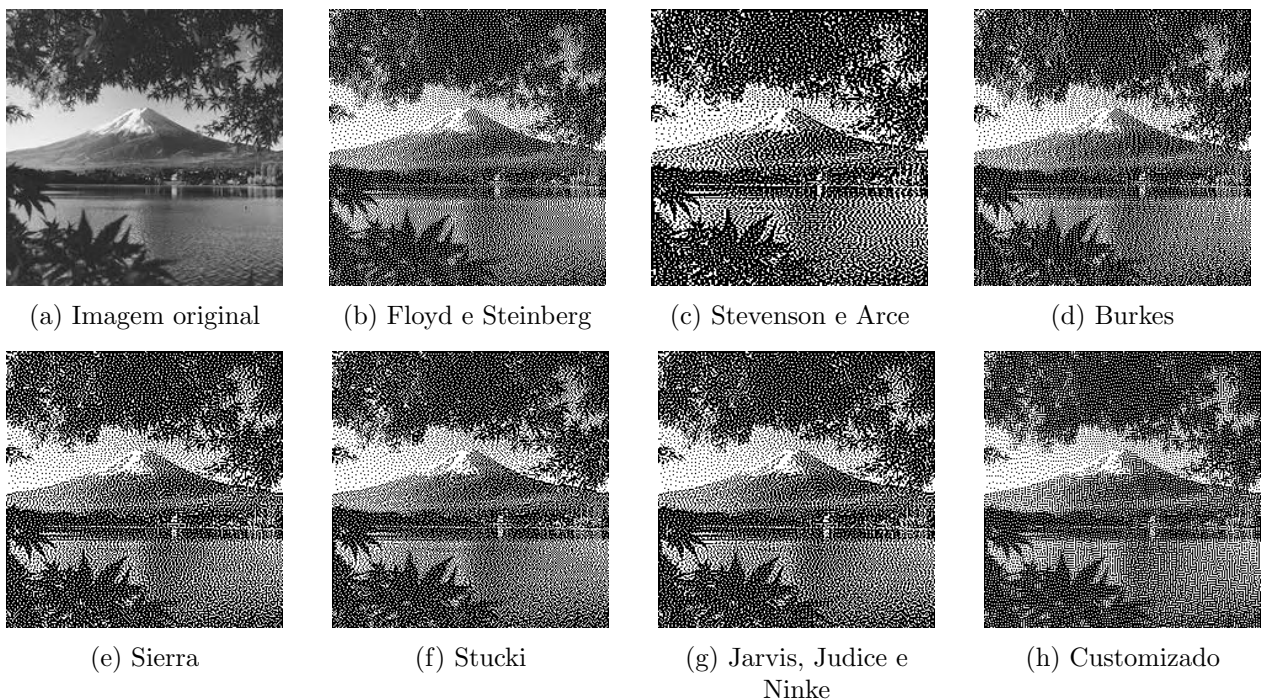


Figura 7 – Resultados da aplicação da técnica de meios-tons com diferentes matrizes de dispersão de erro sobre `fuji_cinza.png` (escala de cinza)

É possível notar, pela Figura 7, que a diferença visual (subjetiva) entre as imagens geradas é pequena, com exceção da matriz customizada e a de Stevenson e Arce. A primeira produz padrões parecidos com “labirintos”, devido à presença de longos caminhos de pixels de mesma cor conectados em vizinhança-4.

A segunda, por outro lado, gera uma imagem onde pixels de mesma cor tendem a ficar mais espaçados e, por conta disso, a textura resultante é visualmente diferente das demais (e, na opinião do autor, mais distante da imagem original), por conta de sua maior dimensão em relação às outras matrizes, fazendo com que padrões regionais - a baixa frequência em uma região, por exemplo - sejam dispersos para pixels distantes. Apesar disso, como se observa na Figura 8 e na Tabela 1, a matriz de Stevenson e Arce apresenta os melhores resultados de acordo com as métricas RMSE, PSNR e correlação, principalmente em imagens pequenas. Portanto, em aplicações de visão computacional e similares, essa pode ser a melhor escolha de técnica de pontilhado.

As outras matrizes utilizadas distribuem o erro em uma região menor do que a matriz de Stevenson e Arce; portanto, não dispersam padrões regionais. Observando a estrutura de cada uma, é possível concluir que as matrizes maiores tendem a gerar transições mais suaves em regiões de alta frequência, pois dispersam mais o erro. Contudo, a diferença visual é quase imperceptível.

Na Figura 8, observa-se uma tendência clara de redução nos valores de correlação com o aumento do tamanho da imagem, independentemente da matriz utilizada. Isso indica que, quanto maior a imagem, menor é a preservação da estrutura original no resultado final, provavelmente devido à dispersão mais ampla do erro.

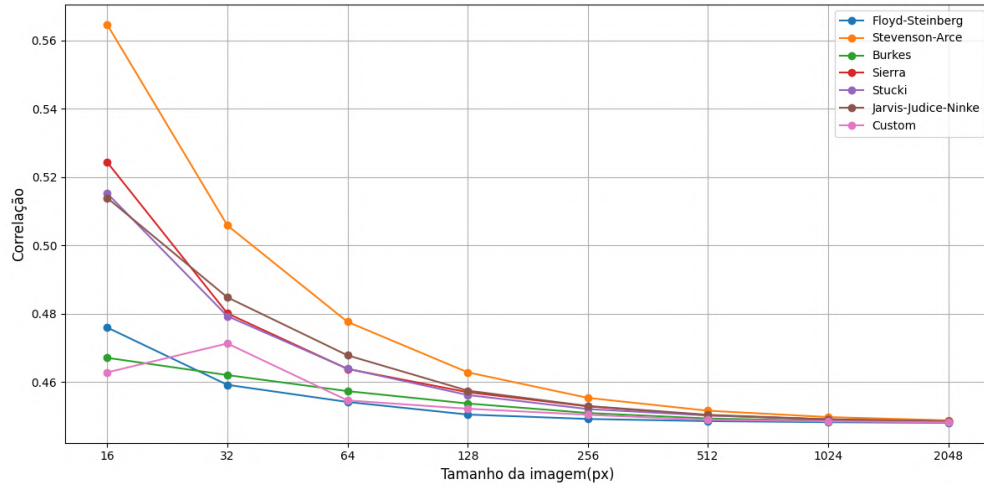


Figura 8 – Correlação vs tamanho da imagem para cada matriz de difusão de erro (as imagens utilizadas para os testes foram iguais a `degrade_radial.png` porém com diferentes tamanhos, geradas através de código, em intensidade de cinza)

Também vale ressaltar que, na distribuição serial dos pesos, o tempo de execução do algoritmo tende a aumentar com o tamanho da matriz. Entretanto, utilizando a implementação paralela, a maior diferença observada foi de 0,16s, entre Floyd-Steinberg e Stevenson-Arce, com uma imagem de 1024x1024 pixels.

Em imagens RGB, como se observa na Figura 10, o efeito visual da aplicação cada matriz é similar ao gerado pela técnica sobre imagens em intensidade de cinza, e as métricas seguem a mesma ordem de qualidade.

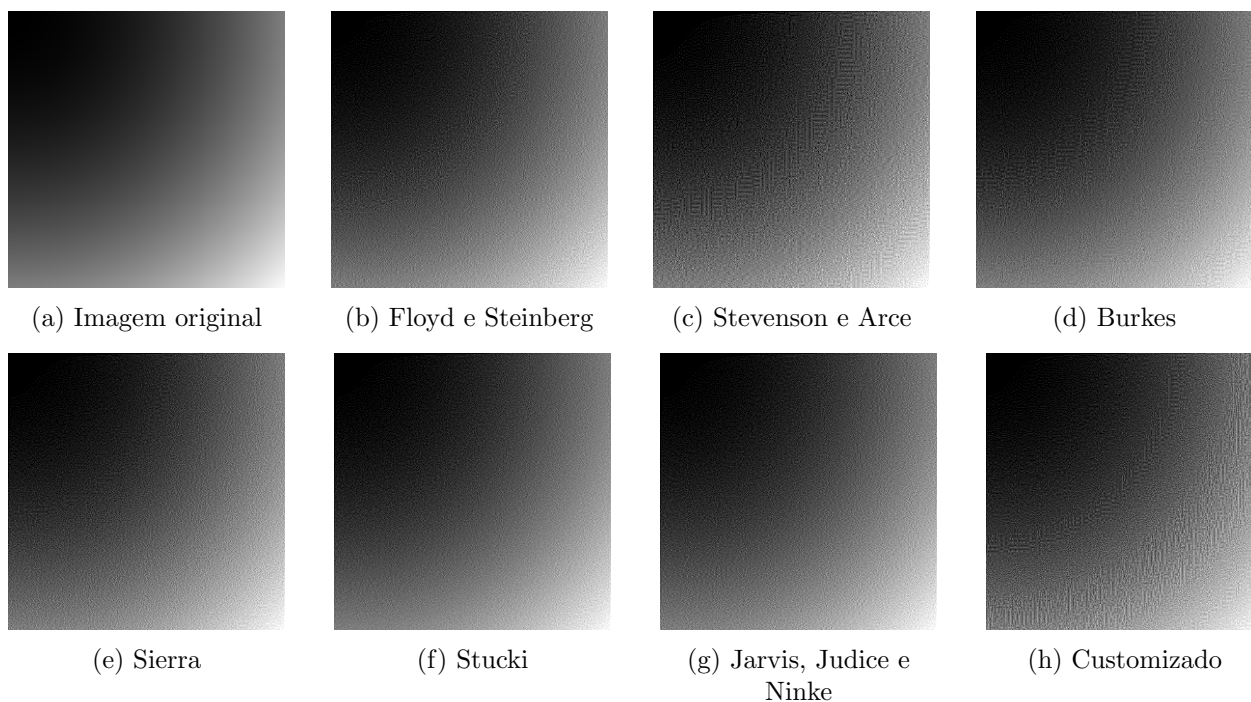


Figura 9 – Resultados da aplicação da técnica de meios-tons com diferentes matrizes de dispersão de erro sobre `degrade_radial.png` (escala de cinza)

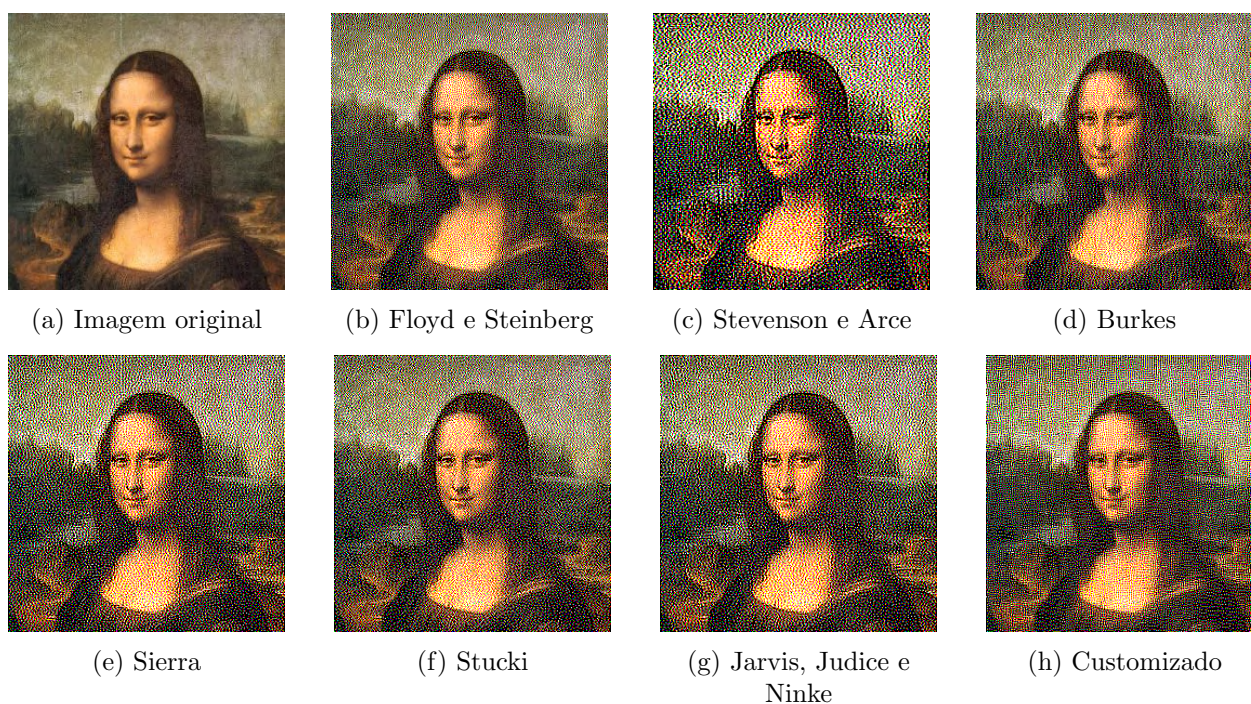


Figura 10 – Resultados da aplicação da técnica de meios-tons com diferentes matrizes de dispersão de erro sobre `monalisa.png` (RGB)

Tabela 1 – Métricas obtidas na aplicação da técnica de meios-tons com diferentes imagens e matrizes de difusão de erro

Imagem	Matriz	RMSE	PSNR	Correlação
fuji_cinza.png (225x225)	Floyd-Steinberg	110.33	7.28	0.4536
	Stevenson-Arce	105.53	7.66	0.5312
	Burkes	109.02	7.38	0.4753
	Sierra	107.38	7.51	0.5020
	Stucki	107.85	7.47	0.4941
	Jarvis-Judice-Ninke	107.15	7.53	0.5054
	Customizado	109.80	7.32	0.4623
monalisa.png (256x256)	Floyd-Steinberg	111.64	7.17	0.4182
	Stevenson-Arce	110.10	7.29	0.4436
	Burkes	111.38	7.19	0.4223
	Sierra	110.78	7.24	0.4325
	Stucki	111.00	7.22	0.4287
	Jarvis-Judice-Ninke	110.67	7.25	0.4343
	Customizado	111.51	7.18	0.4203
degrade_radial.png (1024x1024)	Floyd-Steinberg	107.26	7.52	0.4483
	Stevenson-Arce	107.18	7.53	0.4498
	Burkes	107.23	7.52	0.4487
	Sierra	107.22	7.53	0.4491
	Stucki	107.21	7.53	0.4491
	Jarvis-Judice-Ninke	107.21	7.53	0.4491
	Customizado	107.24	7.52	0.4485

3.5 Limitações

Devido à performance lenta do algoritmo ao processar imagens grandes, a implementação aqui apresentada não é eficiente o bastante para ser utilizada no processamento de vídeos em tempo real (ou aplicações similares, onde várias imagens precisam ser processadas em um curto período de tempo).

Para evidenciar isso, suponha um vídeo RGB com 1280x720 pixels e 60 frames por segundo. Utilizando como base os tempos de execução obtidos na Figura 5, processar cada frame levaria em torno de 8 segundos; dessa forma, processar um segundo do vídeo (60 imagens) levaria 480 segundos de forma serial, e no mínimo 8 segundos em paralelo.

3.6 Conclusão

A técnica de meios-tons por difusão de erro, conforme proposta por Floyd e Steinberg, provou-se eficaz na simulação de níveis intermediários de intensidade utilizando apenas valores binários, tanto para imagens em tons de cinza quanto em RGB. A análise comparativa entre estratégias de varredura diferentes revelou que variações como a varredura em serpentina podem reduzir a geração de padrões indesejados.

No aspecto de implementação, a substituição do *for loop* pela abordagem vetorial via *slicing* mostrou-se vantajosa, especialmente em imagens RGB, permitindo uma aceleração significativa do processamento com perdas numéricas irrelevantes. Ainda assim, limitações fundamentais de paralelização permanecem devido à dependência causal do algoritmo.

Por fim, constatou-se que a utilização de diferentes matrizes de dispersão gera imagens visualmente semelhantes; contudo, dependendo da aplicação, a escolha da matriz deve ser feita considerando também a diferente quantidade de informação sobre a imagem original que elas são capazes de reter.

4 Exercício 2 - Filtragem e compressão no Domínio da Frequência

4.1 Filtragem

4.1.1 Introdução

Filtragem é um conceito essencial no processamento de imagens, podendo ser utilizada em aplicações como atenuação de ruído (com filtros passa-baixas) e realce de bordas (com filtros passa-altas), analisadas no Trabalho 1. No domínio do espaço, uma imagem $f(x, y)$ pode ser filtrada a partir da convolução com um filtro $h(x, y)$, i.e., a imagem filtrada é definida por $g(x, y) = f(x, y) * h(x, y)$.

Em contrapartida, no domínio da frequência, a operação correspondente à convolução no domínio do espaço é a multiplicação. Assim, uma imagem obtida da Transformada de Fourier, $F(u, v) = \mathcal{F}(f(x, y))$, multiplicada por um filtro $H(u, v)$ produz $G(u, v) = F(u, v) \cdot H(u, v)$, de forma que a imagem filtrada no domínio do espaço, $g(x, y)$, pode ser obtida pela Transformada Inversa de Fourier: $g(x, y) = \mathcal{F}^{-1}(G(u, v))$.

Além das aplicações já citadas, a filtragem no domínio do espaço permite selecionar ou remover mais facilmente faixas específicas de frequências, a partir da aplicação de filtros passa-banda e rejeita-banda.

A implementação em código e a análise que segue contemplam os seguintes filtros:

- Ideais: Filtros binários que mantêm inalteradas as frequências desejadas e eliminam todas as outras, multiplicando-as pelos valores 1 e 0, respectivamente, no domínio da frequência.
- Butterworth: Filtros de transição suave, projetado para atenuar gradualmente as frequências fora da faixa desejada. Quanto maior a ordem, mais abrupta é a transição em torno da frequência de corte, aproximando-se de um filtro ideal.
- Gaussianos: Filtros baseados na distribuição normal, que atenuam todas as frequências de forma contínua, proporcionando a suavização mais gradual entre todos os filtros considerados.

4.1.2 Metodologia

A metodologia adotada seguiu a sequência proposta pelo enunciado. Em primeiro lugar, é aplicada a Transformada de Fourier à imagem (através da função do NumPy que realiza a FFT, uma implementação otimizada da transformada) e o espectro da frequência é centralizado. Em seguida, o filtro desejado é criado e multiplicado pela imagem no domínio da frequência. Por fim, a Transformada Inversa de Fourier é aplicada e a imagem filtrada no domínio espacial é recuperada.

Os filtros foram implementados de maneira genérica para permitir parametrização dinâmica através de linha de comando, e tomando proveito do fato de que filtros passa-altas podem ser obtidos a partir da subtração de 1 por um filtro passa-baixa. A mesma relação é válida para filtros passa-banda e rejeita-banda.

Além disso, para permitir a visualização do espectro de magnitude, os pixels $F(u, v)$ do espectro foram representados em decibéis.

4.1.3 Resultados

Nesta seção, será utilizada a imagem `waterfall_cinza.png` para apresentar os resultados. Essa imagem foi escolhida por conter componentes de alta e baixa frequência bem definidas.

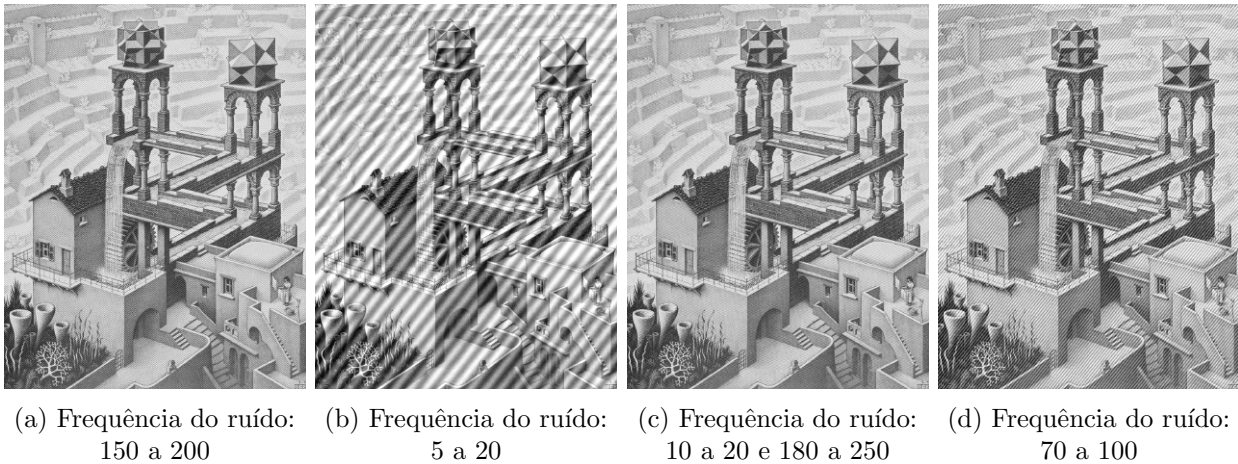


Figura 11 – Imagem `waterfall_cinza.png` (escala de cinza) com diferentes faixas de ruído senoidal acrescentados

Além disso, a fim de demonstrar o efeito de cada filtro, será adicionado à imagem um ruído senoidal de frequência conhecida. Em seguida, os parâmetros dos filtros serão selecionados de modo a remover esse ruído.

1. Filtros passa-baixas - Frequência do ruído: de 150 a 200

Todos os filtros tiveram o efeito de borrar a imagem e removeram o ruído senoidal. Vale notar que o filtro ideal gerou distorções próximo às bordas, o que ocorre devido ao efeito de Gibbs. Isso ocorre pois, no domínio da frequência, esse filtro não é contínuo; assim, a transformada inversa possui uma componente dependente de senoides - principalmente próximo a regiões que antes eram de alta frequência (bordas), que foram removidas.

Além disso, analisando os espectros formados, é possível ver a suavização maior do filtro gaussiano em relação ao Butterworth de ordem 2. Tal fato explica o maior borramento da aplicação daquele em relação a esse.



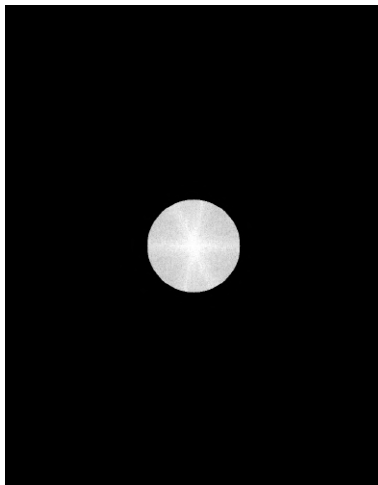
(a) Resultado - Passa-baixas ideal



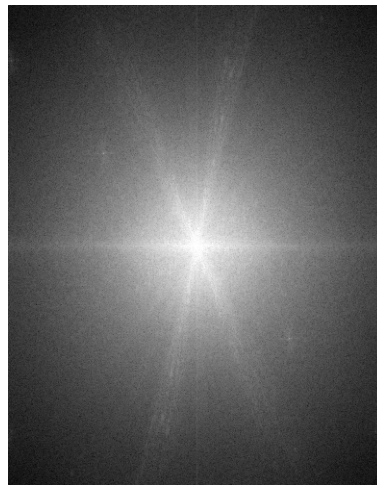
(b) Resultado - Passa-baixas de Butterworth de 2ª ordem



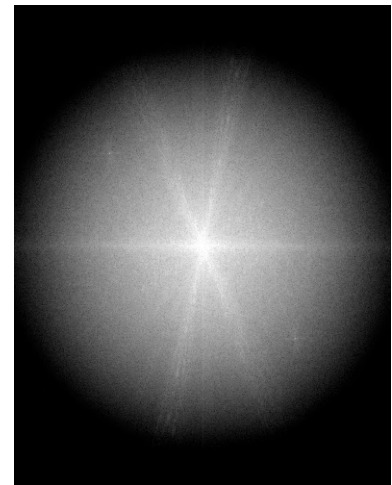
(c) Resultado - Passa-baixas gaussiano



(d) Espectro de Fourier (magnitude) - Passa-baixas ideal



(e) Espectro de Fourier (magnitude) - Passa-baixas de Butterworth de 2ª ordem



(f) Espectro de Fourier (magnitude) - Passa-baixas gaussiano

Figura 12 – Resultados da aplicação dos filtros passa-baixas com frequência de corte igual a 100

2. Filtros passa-altas - Frequência do ruído: de 5 a 20

Como esperado, os filtros passa-altas realçaram as bordas da imagem, e foram capazes de remover o ruído de baixa frequência. Nota-se que o destaque das bordas é mais perceptível na aplicação do filtro ideal - pois esse não mantém as baixas frequências -, o que pode ser observado pelas “sombras” claras mais destacadas nesse caso. Contudo, a olho nu, os resultados apresentam grande similaridade visual.

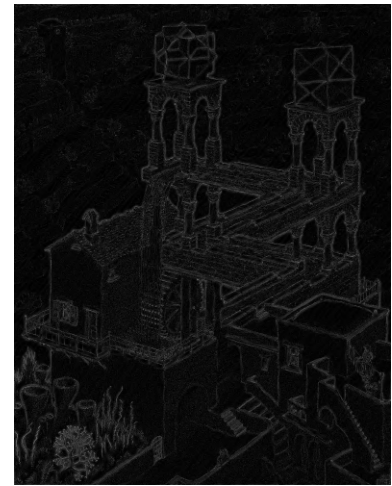
Apesar disso, os espectros gerados mostram que o filtro gaussiano apenas filtrou totalmente as frequências em uma pequena proximidade do ponto central.



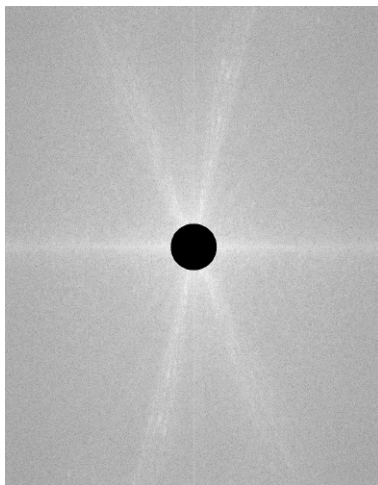
(a) Resultado - Passa-altas ideal



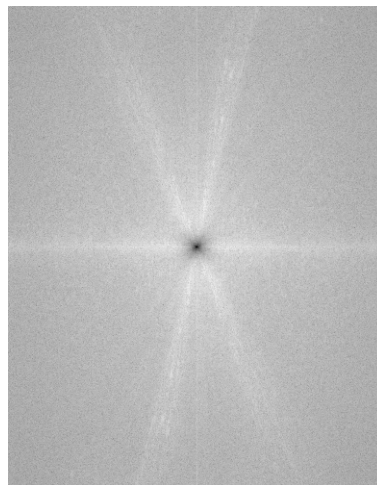
(b) Resultado - Passa-altas de Butterworth de 2ª ordem



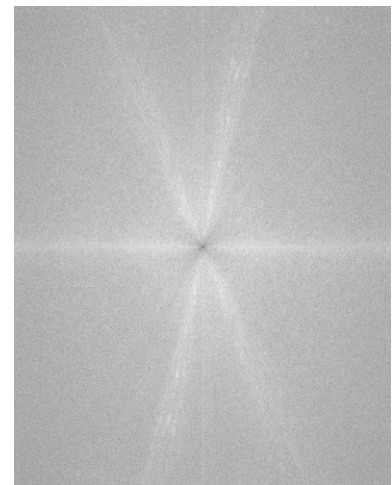
(c) Resultado - Passa-altas gaussiano



(d) Espectro de Fourier (magnitude) - Passa-altas ideal



(e) Espectro de Fourier (magnitude) - Passa-altas de Butterworth de 2ª ordem



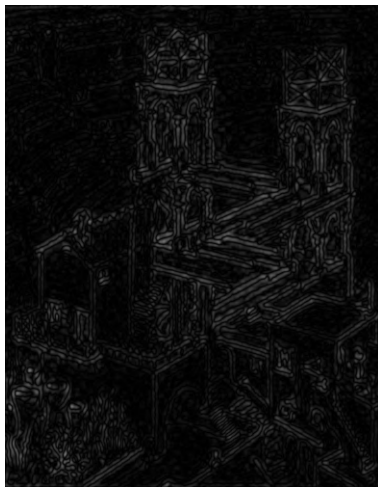
(f) Espectro de Fourier (magnitude) - Passa-altas gaussiano

Figura 13 – Resultados da aplicação dos filtros passa-altas com frequência de corte igual a 50

3. Filtros passa-faixa - Frequência do ruído: de 10 a 20 e de 180 a 250

Destarte, todos os resultados gerados pela aplicação dos filtros passa-faixa foram capazes de remover o ruído senoidal. Os filtros ideal e de Butterworth geraram resultados similares, com a diferença de aquele apresentou maior distorção nas bordas - devido ao efeito de Gibbs, citado anteriormente -, que esse. Isso se dá pois a faixa escolhida possui tantas frequências baixas quanto altas.

Entretanto, o filtro gaussiano gerou um resultado muito diferente dos outros. Nota-se, pelo espectro de Fourier gerado, que esse filtro agiu de forma similar a um passa-baixas na região aplicada, o que explica o efeito de borradura, mas com pouca atenuação das bordas.



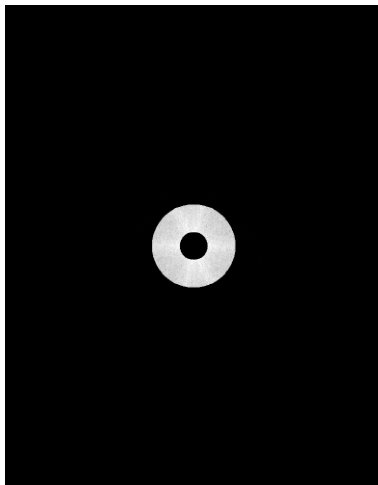
(a) Resultado - Passa-faixa ideal



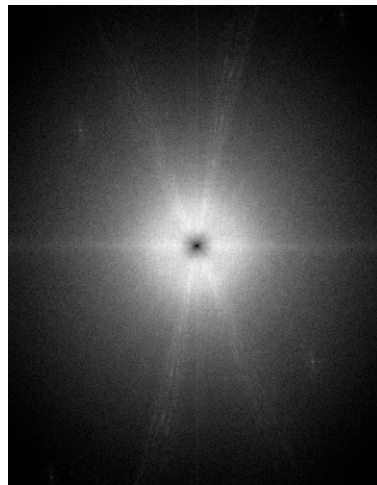
(b) Resultado - Passa-faixa de Butterworth de 2ª ordem



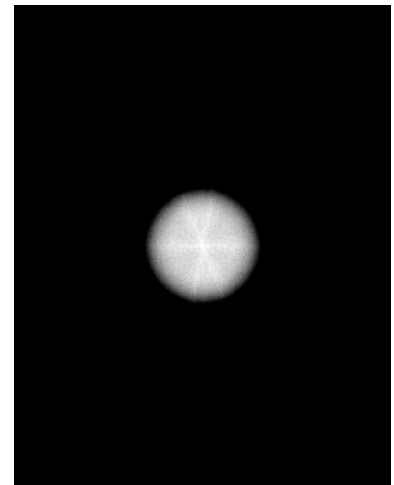
(c) Resultado - Passa-faixa gaussiano



(d) Espectro de Fourier (magnitude) - Passa-faixa ideal



(e) Espectro de Fourier (magnitude) - Passa-faixa de Butterworth de 2ª ordem



(f) Espectro de Fourier (magnitude) - Passa-faixa gaussiano

Figura 14 – Resultados da aplicação dos filtros passa-faixa com frequência de corte e largura iguais a 60

4. Filtros rejeita-faixa - Frequência do ruído: de 70 a 100

Novamente, todos os filtros foram capazes de remover o ruído senoidal. Porém, aqui tiveram o efeito de borrar a imagem sem atenuação severa das bordas.

O filtro com maior efeito de borramento foi o ideal, com introdução de distorções próximo às bordas devido ao efeito de Gibbs, seguido do filtro de Butterworth e, depois, do gaussiano - esse último atenuou menos as bordas.

Observa-se, nos espectros, que a faixa negada completamente pelo filtro gaussiano foi menor. Isso, aliado à sua atenuação mais suave, explicam os fatos citados.



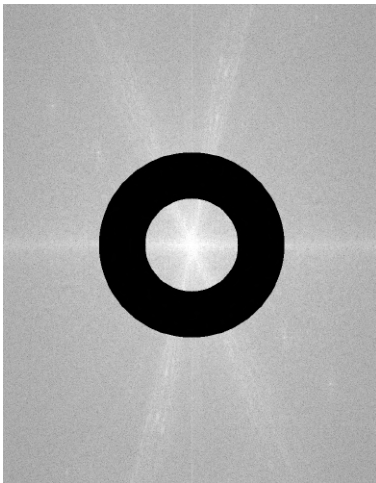
(a) Resultado - Rejeita-faixa ideal



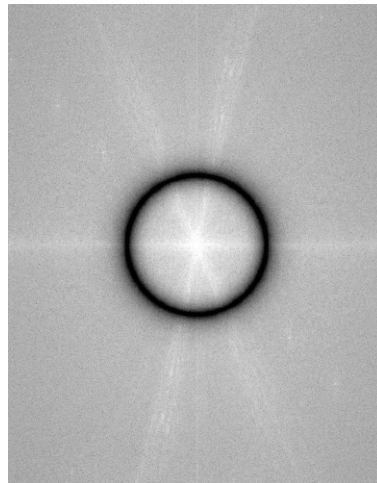
(b) Resultado - Rejeita-faixa de Butterworth de 2ª ordem



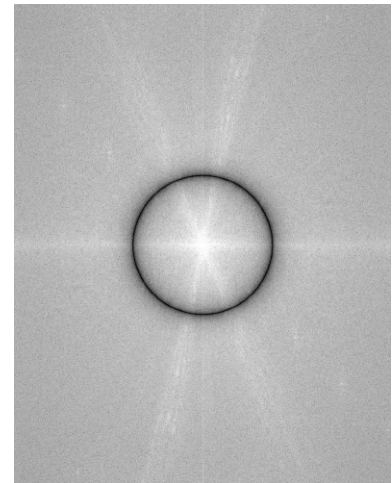
(c) Resultado - Rejeita-faixa gaussiano



(d) Espectro de Fourier (magnitude) - Rejeita-faixa ideal



(e) Espectro de Fourier (magnitude) - Rejeita-faixa de Butterworth de 2ª ordem



(f) Espectro de Fourier (magnitude) - Rejeita-faixa gaussiano

Figura 15 – Resultados da aplicação dos filtros rejeita-faixa com frequência de corte e largura iguais a 150 e 100, respectivamente

Em todos os casos, o filtro de Butterworth foi utilizado com ordem 2 pois, para valores intermediários (3 a 5), ele é similar ao gaussiano; em contrapartida, com valores maiores, passa se aproximar do filtro ideal, como se observa na Figura 16.

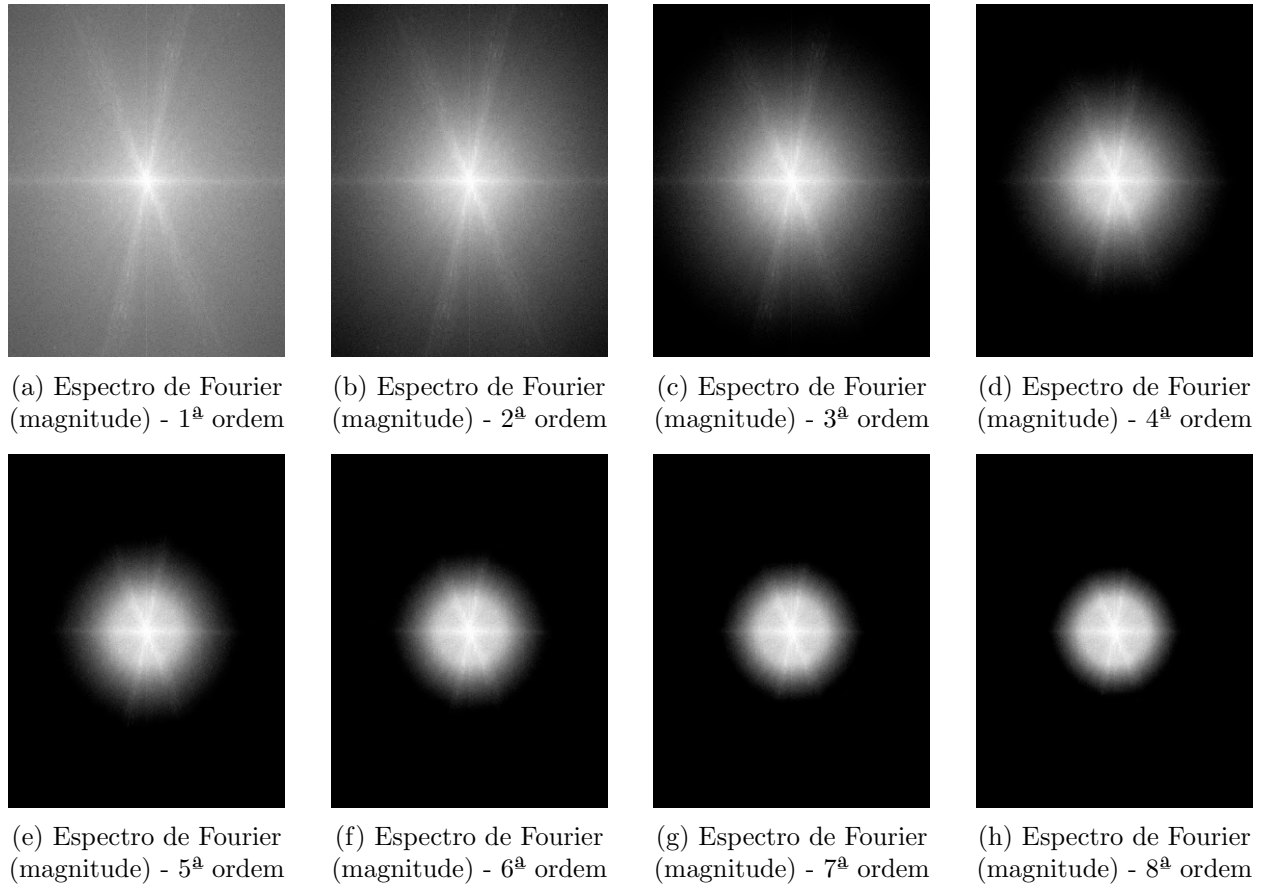


Figura 16 – Espectros de magnitude obtidos com o filtro de Butterworth passa-baixas de diferentes ordens. Todos possuem frequência de corte igual a 100.

4.1.4 Limitações

Como observado na seção 4.1.3, o filtro ideal, por realizar o corte abrupto de frequências, pode introduzir distorções ao resultado devido ao efeito de Gibbs. Porém, esse efeito é menos perceptível quando um filtro ideal que atenua baixas frequências é utilizado. Os outros filtros também introduzem artefatos, embora em menor intensidade.

A implementação em código, nesse exercício, não possibilita o processamento de imagens multibanda, o que poderia ser feito alterando-o para realizar a filtragem em cada banda.

Por fim, como a Transformada Rápida de Fourier (FFT) e sua inversa em duas dimensões possuem complexidade $O(MN \log MN)$, onde M e N são as dimensões da imagem, a filtragem pode ser computacionalmente custosa para imagens muito grandes. No entanto, os resultados aqui apresentados foram gerados rapidamente: o processamento completo levou apenas 0,71 segundos, mesmo considerando que a imagem `waterfall_cinza.png` possui dimensões de 813×1038 .

4.2 Compressão

4.2.1 Introdução

A compressão de imagens é um processo que visa reduzir a quantidade de dados necessários para representá-las, mantendo a maior qualidade visual possível. No domínio da frequência, a compressão tem base no fato de que grande parte da informação de uma imagem está concentrada em valores baixos de frequência, indicado pela grande magnitude atrelada a essas.

Esta abordagem é útil para reduzir o tamanho de armazenamento de imagens, como será visto a seguir.

4.2.2 Metodologia

A compressão foi feita de forma análoga à filtragem em frequência, com a exceção de que, em vez de aplicar um filtro para cortar frequências as indesejadas, aqui é feita a remoção das frequências cuja magnitude seja inferior a um valor de percentil.

O uso do percentil como critério de corte permite ajustar o nível de compressão de forma intuitiva: percentis baixos preservam mais informações de alta frequência, enquanto percentis altos resultam em maior perda de detalhes.

4.2.3 Resultados

Na Figura 18 é possível observar os resultados da compressão para a imagem `baboon_monocromatica.png` e diferentes valores de percentil.

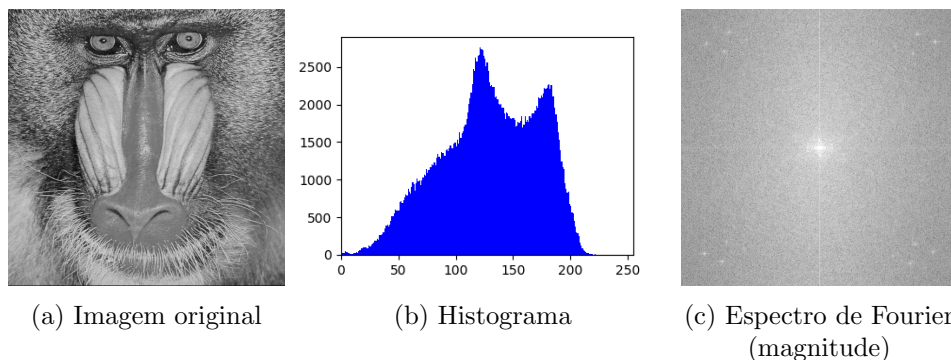


Figura 17 – Dados da imagem `baboon_monocromatica.png` antes da compressão.

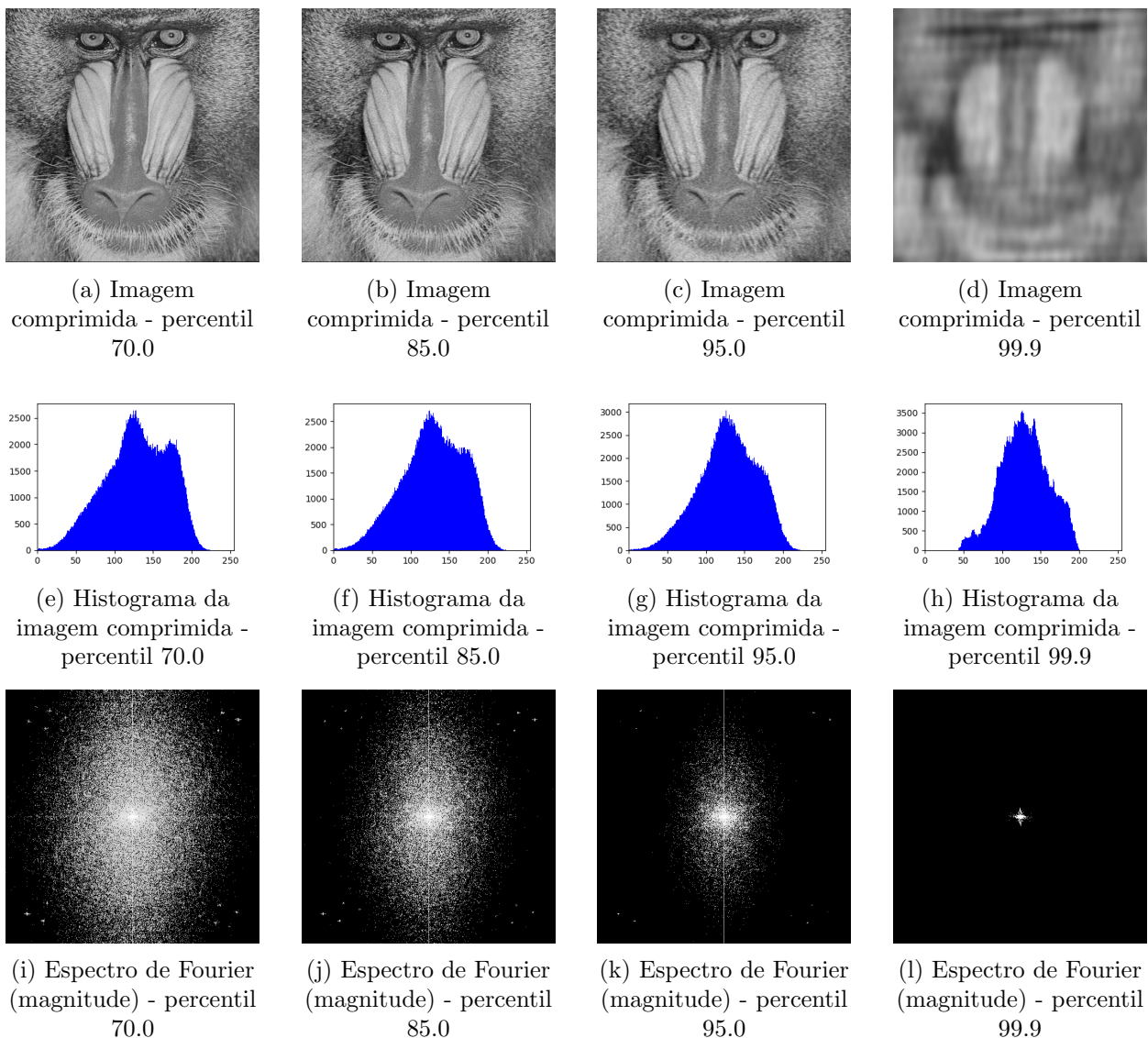


Figura 18 – Resultados da aplicação dos filtros passa-faixa com frequência de corte e largura iguais a 60

Note que, quanto maior é o percentil de frequências removidas, mais borrada se torna a imagem. Tal fato pode ser explicado pelo efeito dessa remoção no espectro de magnitude de Fourier: quando o percentil aumenta, a tendência é que frequências altas sejam perdidas.

Além disso, é possível observar que essa imagem possui alguns pontos de alta frequência - presentes nas pontas do espectro - que só são removidos, nas imagens mostradas, no percentil 99.9, imagem na qual as bordas têm alto grau de borramento, i.e., esses poucos pontos representam a maior parte da informação de alta frequência da imagem.

Vale ressaltar, também, que a imagem comprimida com percentil 70.0 é visualmente similar à imagem original, e as com percentil 85.0 e 95.0 possuem um borramento leve, ainda sendo fácil reconhecer a imagem. Por outro lado, a imagem de maior compressão apresenta um alto borramento, embora ainda seja reconhecível.

Nota-se que o histograma da imagem comprimida tende a se estreitar, indicando uma diminuição do contraste da imagem com o aumento do percentil.

Por fim, é importante citar a diminuição nos tamanhos das imagens causado pela compressão. A imagem original possui 233,12 KB, e, na sequência de aumento de percentil, o tamanho se reduz para 203,40, 194,02, 174,67 e 77,24. Note que essa sequência não apresenta relação linear com o percentil - a maior compressão ocorre de 95.0 para 99.9 -, o que ocorre pois, quanto maior a magnitude de um pixel removido do espectro de frequência, maior a informação da imagem que é perdida.

4.2.4 Limitações

Novamente, o código só está definido para a escala de cinza. Além disso, o critério de compressão leva em conta apenas o valor de magnitude em frequência; uma versão mais elaborada poderia considerar outros critérios, como a posição dos pixels no espectro.

4.3 Conclusão

Esse exercício permitiu aplicar e analisar conceitos fundamentais de processamento de imagens no domínio da frequência: a filtragem e a compressão.

Na etapa de filtragem, observou-se que filtros ideais, de Butterworth e gaussianos apresentam comportamentos distintos, principalmente na transição entre frequências mantidas e atenuadas. Os filtros ideais, apesar de selecionarem uma faixa de forma precisa, apresentam maior tendência a introduzir distorções próximo às bordas devido ao efeito de Gibbs. Em contrapartida, os outros filtros, por possuírem transições mais suaves, atenuam essas distorções.

Já na etapa de compressão, foi demonstrado que a maior parte da informação de uma imagem tende a estar concentrada em baixas frequências. Dessa forma, a remoção de frequências de baixa magnitude permitiu reduzir a quantidade de dados, mantendo qualidade visual próxima até certos limites de percentil.

5 Referências

- [1] Contribuidores da Wikipedia. *Floyd–Steinberg dithering*. Disponível em: https://en.wikipedia.org/wiki/Floyd%E2%80%93Steinberg_dithering. Acesso em: 28 abr. 2025.
- [2] GUILLOTEAU, Quentin. *Parallel Dithering: How Fast Can We Go?*. 2022. Disponível em: https://guilloteau.github.io/downloads/Parallel_Dithering_Quentin_Guilloteau.pdf. Acesso em: 28 abr. 2025.
- [3] NV5 Geospatial Software. *BANDPASS_FILTER*. Disponível em: https://www.nv5geospatialsoftware.com/docs/BANDPASS_FILTER.html. Acesso em: 28 abr. 2025.